In [7]:

```
# EX3a: MODEL PLANNING AND BUILDING - CLASSIFICATION
# Customer Churn Prediction


# ============================================================
# 1. USE CustomerChurn_Preprocessed.csv FILE
# ============================================================

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)

# Load the preprocessed dataset
df = pd.read_csv("customer_churn_dataset-testing-master.csv")

print("Dataset:")
print(df.head())

print("\nDataset Shape:", df.shape)


# ============================================================
# 2. SPLIT DATASET INTO TRAINING (80%) AND TESTING (20%)
# ============================================================

# Remove CustomerID because it is only an identifier
df = df.drop("CustomerID", axis=1)

# Separate input features and target variable
X = df.drop("Churn", axis=1)
y = df["Churn"]

# Identify categorical columns
categorical_columns = [
    "Gender",
    "Subscription Type",
    "Contract Length"
]

# Preprocess categorical columns using One-Hot Encoding
preprocessor = ColumnTransformer(
```

```
    transformers=[
        (
            "categorical",
            OneHotEncoder(handle_unknown="ignore"),
            categorical_columns
        )
    ],
    remainder="passthrough"
)

# Split into 80% training and 20% testing
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nTraining Data:", X_train.shape)
print("Testing Data:", X_test.shape)


# ============================================================
# 3. TRAIN LOGISTIC REGRESSION AND PREDICT CHURN
# ============================================================

logistic_model = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        ("classifier", LogisticRegression(max_iter=1000))
    ]
)

# Train Logistic Regression
logistic_model.fit(X_train, y_train)

# Predict customer churn
y_pred_lr = logistic_model.predict(X_test)

print("\nLogistic Regression predictions:")
print(y_pred_lr[:10])


# ============================================================
# 4. TRAIN DECISION TREE AND PREDICT CHURN
# ============================================================

decision_tree_model = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        ("classifier", DecisionTreeClassifier(random_state=42))
    ]
)

# Train Decision Tree
decision_tree_model.fit(X_train, y_train)

# Predict customer churn
y_pred_dt = decision_tree_model.predict(X_test)
```

```
print("\nDecision Tree predictions:")
print(y_pred_dt[:10])


# ============================================================
# 5. TRAIN RANDOM FOREST AND PREDICT CHURN
# ============================================================

random_forest_model = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        (
            "classifier",
            RandomForestClassifier(
                n_estimators=100,
                random_state=42
            )
        )
    ]
)

# Train Random Forest
random_forest_model.fit(X_train, y_train)

# Predict customer churn
y_pred_rf = random_forest_model.predict(X_test)

print("\nRandom Forest predictions:")
print(y_pred_rf[:10])


# ============================================================
# 6. EVALUATE EACH MODEL
#    Confusion Matrix, Accuracy, Precision, Recall, F1-Score
# ============================================================

def evaluate_model(y_test, y_pred):

    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(
        y_test, y_pred, zero_division=0
    )
    recall = recall_score(
        y_test, y_pred, zero_division=0
    )
    f1 = f1_score(
        y_test, y_pred, zero_division=0
    )

    return accuracy, precision, recall, f1


# Calculate metrics
lr_metrics = evaluate_model(y_test, y_pred_lr)
dt_metrics = evaluate_model(y_test, y_pred_dt)
rf_metrics = evaluate_model(y_test, y_pred_rf)


# Create comparison table
```

```python
results = pd.DataFrame(
    [lr_metrics, dt_metrics, rf_metrics],
    columns=[
        "Accuracy",
        "Precision",
        "Recall",
        "F1-Score"
    ],
    index=[
        "Logistic Regression",
        "Decision Tree",
        "Random Forest"
    ]
)

print("\n================================================")
print("MODEL EVALUATION RESULTS")
print("================================================")

print(results.round(4))


# ---------------- CONFUSION MATRIX ----------------

# Logistic Regression
cm_lr = confusion_matrix(y_test, y_pred_lr)

plt.figure(figsize=(5, 4))
sns.heatmap(
    cm_lr,
    annot=True,
    fmt="d",
    cmap="Blues"
)
plt.title("Logistic Regression - Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()


# Decision Tree
cm_dt = confusion_matrix(y_test, y_pred_dt)

plt.figure(figsize=(5, 4))
sns.heatmap(
    cm_dt,
    annot=True,
    fmt="d",
    cmap="Blues"
)
plt.title("Decision Tree - Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()


# Random Forest
cm_rf = confusion_matrix(y_test, y_pred_rf)

plt.figure(figsize=(5, 4))
```

```
sns.heatmap(
    cm_rf,
    annot=True,
    fmt="d",
    cmap="Blues"
)
plt.title("Random Forest - Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()


# ============================================================
# 7. SINGLE BAR CHART FOR PERFORMANCE COMPARISON
# ============================================================

results.plot(
    kind="bar",
    figsize=(10, 6)
)

plt.title("Comparison of Classification Models")
plt.xlabel("Models")
plt.ylabel("Score")
plt.ylim(0, 1)
plt.xticks(rotation=0)
plt.legend(title="Evaluation Metrics")
plt.grid(
    axis="y",
    linestyle="--",
    alpha=0.7
)

plt.tight_layout()
plt.show()


# ============================================================
# 8. IDENTIFY THE BEST-PERFORMING MODEL
# ============================================================

# F1-Score is used because it provides a balance between
# Precision and Recall.

best_model = results["F1-Score"].idxmax()
best_f1_score = results["F1-Score"].max()

print("\n================================================")
print("BEST-PERFORMING MODEL")
print("================================================")

print("Best Model:", best_model)
print("F1-Score:", round(best_f1_score, 4))

print("\nComplete Performance:")
print(results.round(4))


# ============================================================
# 9. BUSINESS RECOMMENDATIONS
```

```
# ============================================================

print("\n================================================")
print("BUSINESS RECOMMENDATIONS")
print("================================================")

print("\n1. Target High-Risk Customers:")
print(
    "Use the best-performing churn prediction model to identify "
    "customers who are most likely to leave and provide them with "
    "personalized retention offers and discounts."
)

print("\n2. Improve Customer Service:")
print(
    "Monitor customers with frequent support calls or service "
    "issues and provide faster support, better complaint resolution, "
    "and proactive customer assistance."
)

print("\n3. Provide Personalized Subscription Plans:")
print(
    "Offer flexible plans, loyalty benefits, customized pricing, "
    "and suitable contract options to customers who have a high "
    "risk of churn."
)
```

```
Dataset:
   CustomerID  Age  Gender  Tenure  Usage Frequency  Support Calls  \
0           1   22  Female      25               14              4
1           2   41  Female      28               28              7
2           3   47    Male      27               10              2
3           4   35    Male       9               12              5
4           5   53  Female      58               24              9

   Payment Delay Subscription Type Contract Length  Total Spend  \
0             27             Basic         Monthly          598
1             13          Standard         Monthly          584
2             29           Premium          Annual          757
3             17           Premium       Quarterly          232
4              2          Standard          Annual          533

   Last Interaction  Churn
0                 9      1
1                20      0
2                21      0
3                18      0
4                18      0

Dataset Shape: (64374, 12)

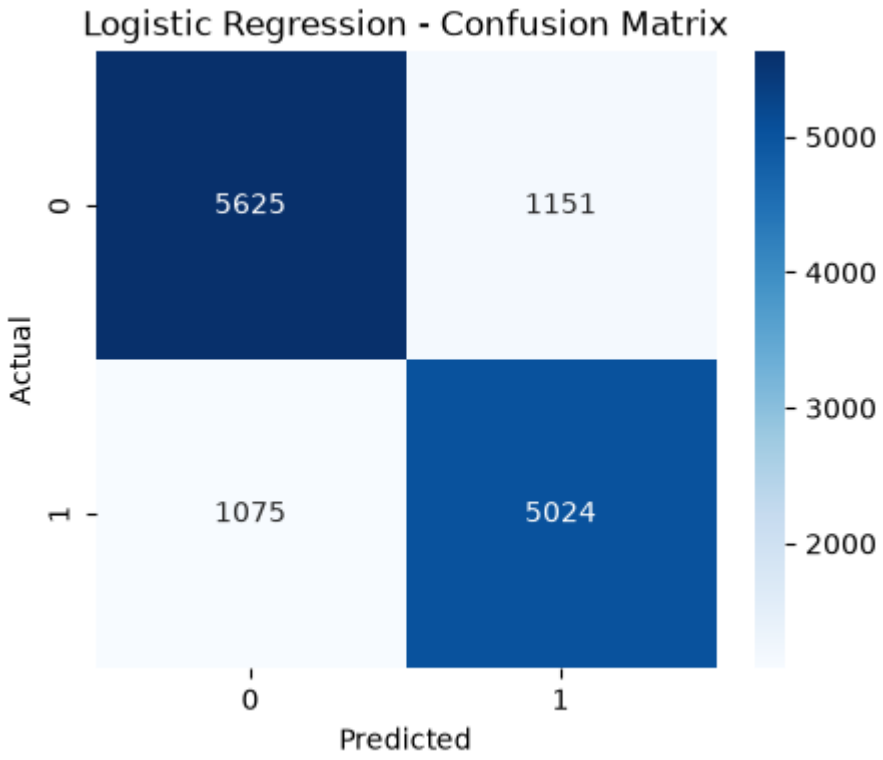
Training Data: (51499, 10)
Testing Data: (12875, 10)

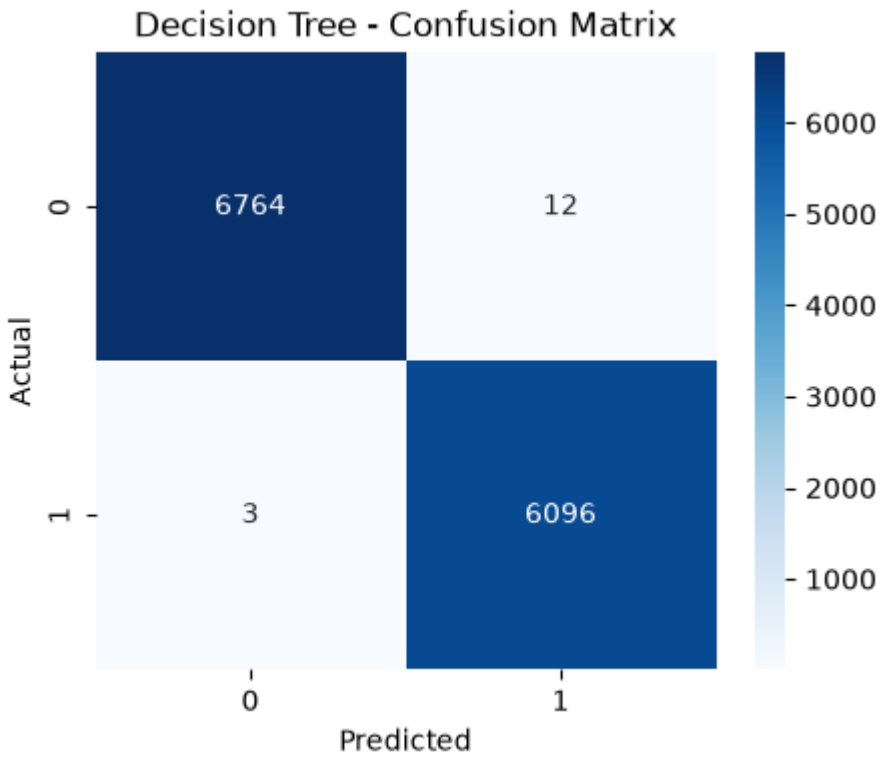
Logistic Regression predictions:
[0 0 0 0 0 0 0 0 0 0]

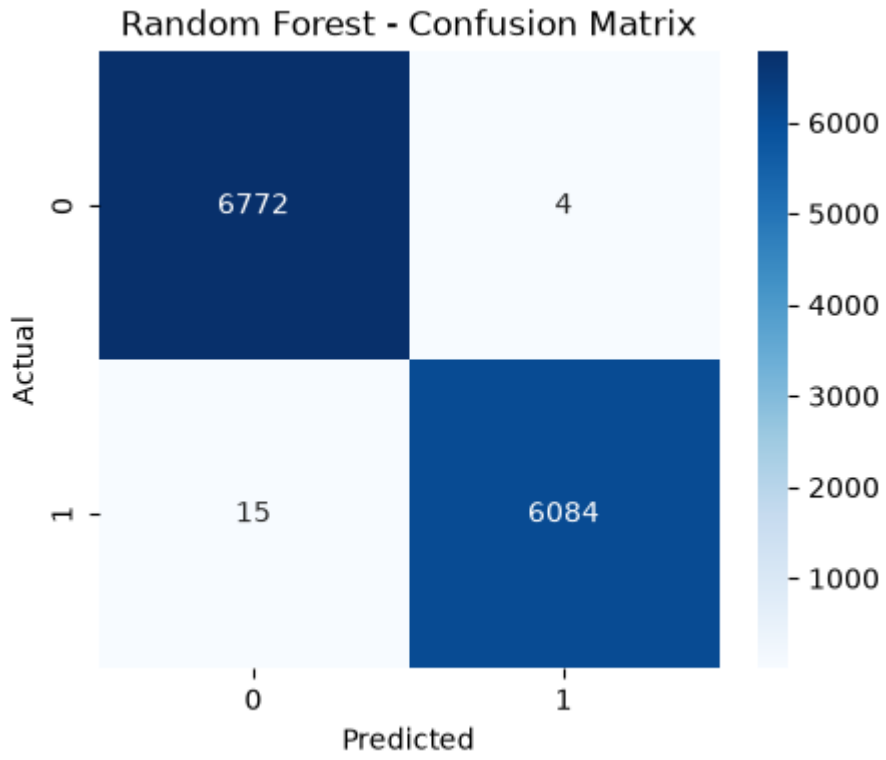
Decision Tree predictions:
[0 1 0 0 0 0 0 0 0 0]

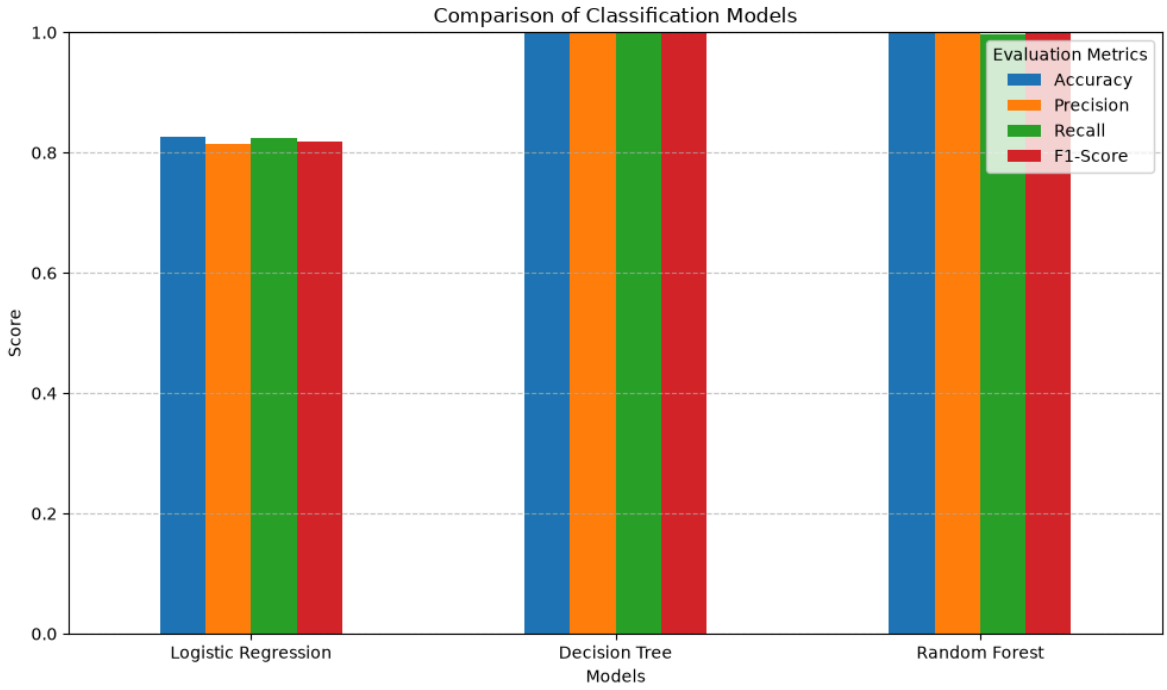
Random Forest predictions:
[0 1 0 0 0 0 0 0 0 0]

================================================
MODEL EVALUATION RESULTS
================================================
                     Accuracy  Precision  Recall  F1-Score
Logistic Regression    0.8271     0.8136  0.8237    0.8186
Decision Tree          0.9988     0.9980  0.9995    0.9988
Random Forest          0.9985     0.9993  0.9975    0.9984
```


Logistic Regression - Confusion Matrix
5625
1151
1075
5024
Actual
0
1
Predicted
0
1
5000
4000
3000
2000


Decision Tree - Confusion Matrix
6764
12
3
6096
Actual
0
1
Predicted
0
1
6000
5000
4000
3000
2000
1000


Random Forest - Confusion Matrix
6772
4
15
6084
Actual
Predicted
0
1
6000
5000
4000
3000
2000
1000


Comparison of Classification Models
Evaluation Metrics
Accuracy
Precision
Recall
F1-Score
Score
1.0
0.8
0.6
0.4
0.2
0.0
Logistic Regression
Decision Tree
Random Forest
Models

```
================================================
BEST-PERFORMING MODEL
================================================
Best Model: Decision Tree
F1-Score: 0.9988

Complete Performance:
                     Accuracy  Precision  Recall  F1-Score
Logistic Regression    0.8271     0.8136  0.8237    0.8186
Decision Tree          0.9988     0.9980  0.9995    0.9988
Random Forest          0.9985     0.9993  0.9975    0.9984

================================================
BUSINESS RECOMMENDATIONS
================================================

1. Target High-Risk Customers:
Use the best-performing churn prediction model to identify customers who are most
likely to leave and provide them with personalized retention offers and discount
s.

2. Improve Customer Service:
Monitor customers with frequent support calls or service issues and provide faste
r support, better complaint resolution, and proactive customer assistance.

3. Provide Personalized Subscription Plans:
Offer flexible plans, loyalty benefits, customized pricing, and suitable contract
options to customers who have a high risk of churn.
```

In [ ]: